

LLMs

Language Models

Course Contents

Deep Learning

Transformer

LLMs

Context Engineering

Finetuning

Instruction Tuning

Agents

The Bigger Picture

This Part: LLMs

- encoder and decoder LLMs
- scaling laws
- LLM zoo

Transformer Architectures for LLMs

encoder-decoder LLMs: sequence-to-sequence, e.g., machine translation

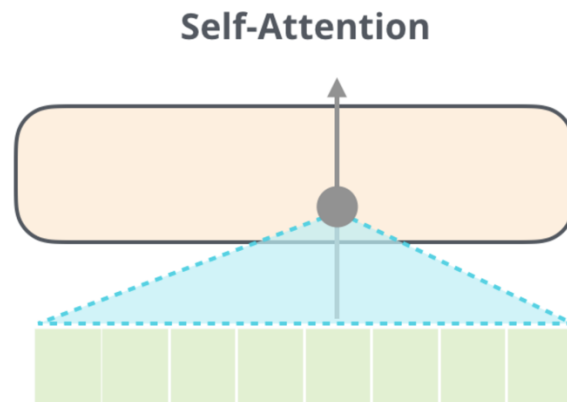
encoder-only LLMs:

- learning of contextual embeddings (for subsequent fine-tuning)
- training: prediction of masked words
- can't generate text, can't be prompted

decoder-only LLMs:

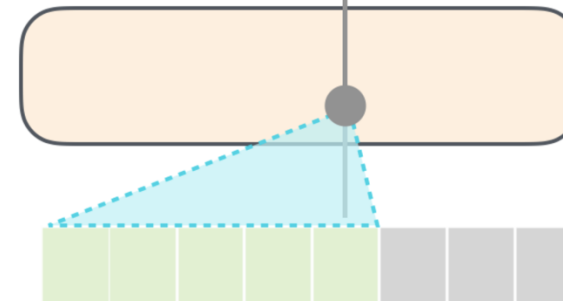
- text generation: e.g., chat bot (after instruction tuning)
- training: next-word prediction
- output one token at a time (auto-regressive: consuming its own output, potentially starting with prompts)

example:
BERT



source

Masked Self-Attention
aka causal



example:
GPT

Encoder-Decoder Attention

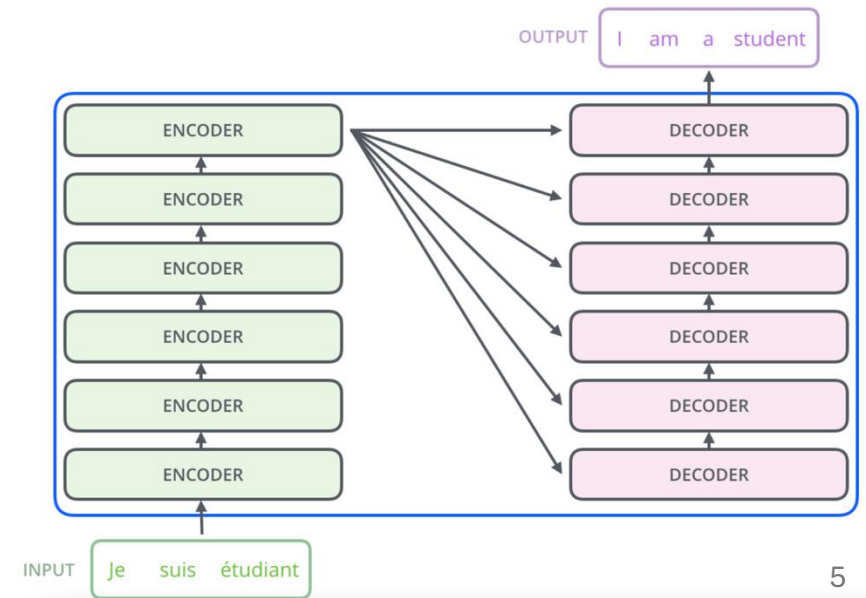
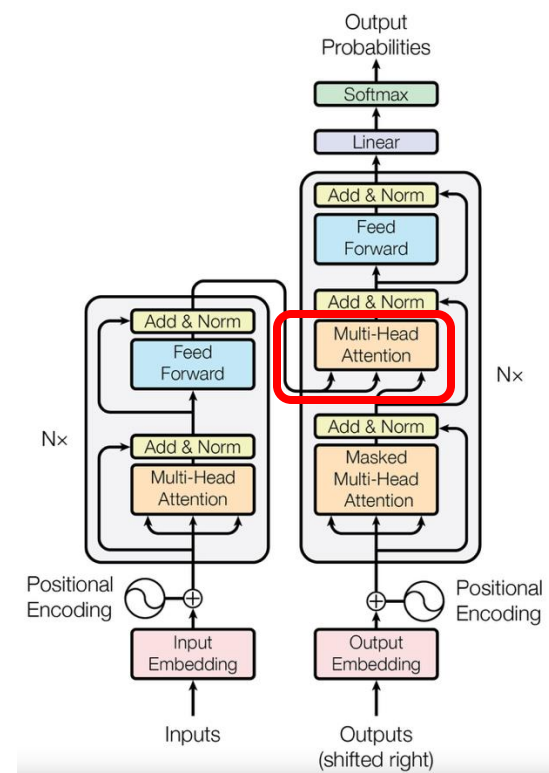
popular use case: machine translation

aka cross-attention:

connection between encoder and decoder,
helping decoder to focus on relevant parts
of input sentence

output of last encoder transformed into K
and V

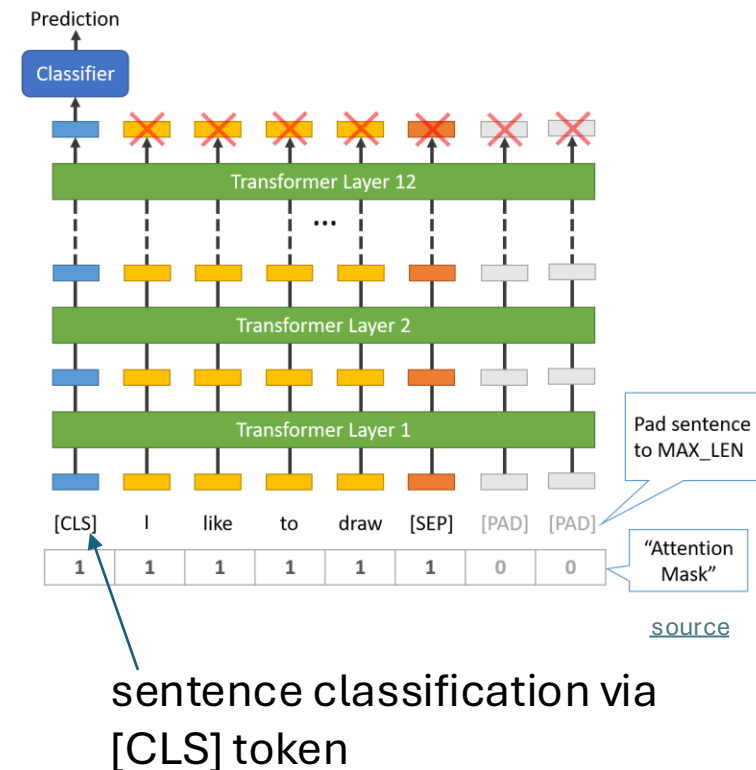
Q from each decoder's cross-attention layer



Example for Encoder-Only LLM

BERT (Bidirectional Encoder Representations from Transformers, Google):

- stack of transformer encoders
- outputting representation (contextual embeddings) to be used/finetuned in specific tasks and data sets (e.g., text classification)
- bidirectional: jointly conditioning on both left and right context
- pre-trained in self-supervised manner on massive data sets
 - masked tokens to be predicted from context
 - next sentence prediction



BERT Variants

BERT: effective but computationally heavy (large models, slow inference)

RoBERTa: even better than BERT but still heavy

- removed next sentence prediction as pretraining objective
- ~ten times more training data, longer training, larger batches, longer sequences
- dynamic masking: instead of fixed masked tokens for each sequence, masks change each epoch

DistilBERT: much lighter and faster, retaining ~95% of BERT's performance

- a smaller “student” model is trained to mimic BERT's predictions
- 40% fewer parameters than BERT, 60% faster inference, using half the memory
- also removed next sentence prediction as pretraining objective

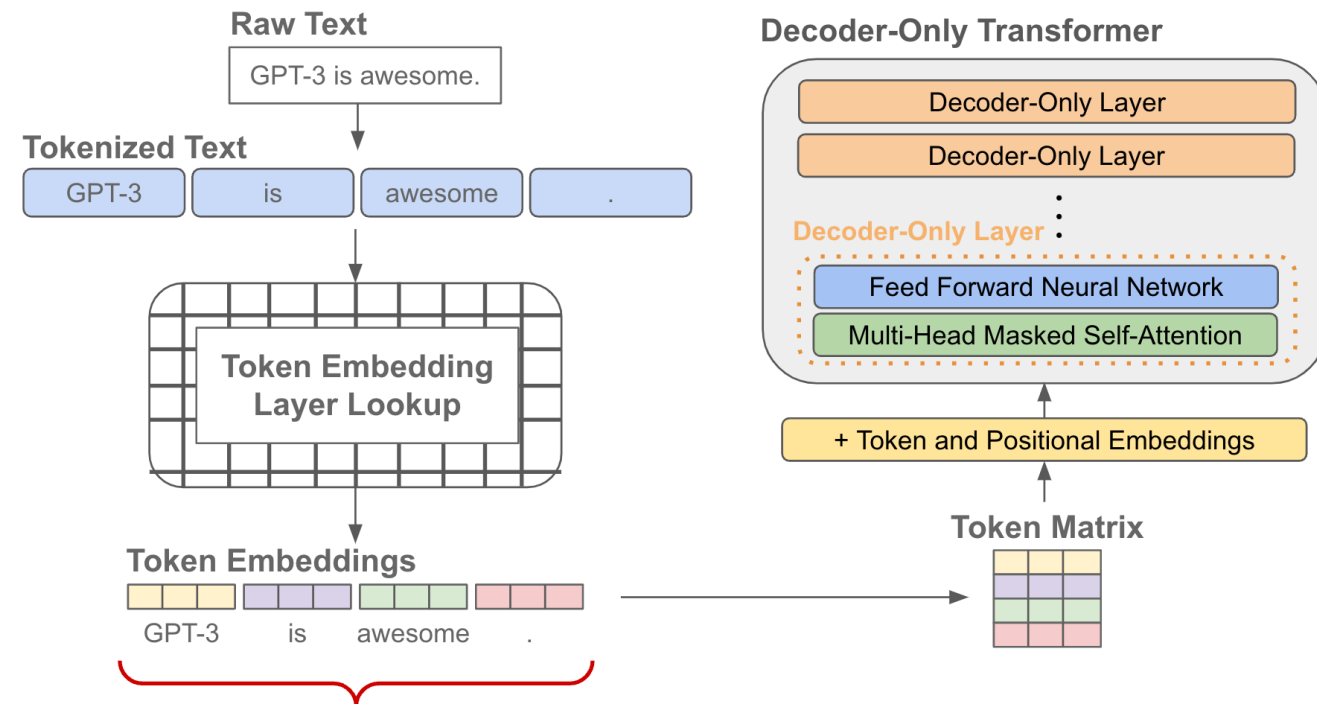
Example for Decoder-Only LLM

GPT (Generative Pre-trained Transformer, by OpenAI) series:

- stack of transformer decoders
→ auto-regressive language model
- generative pre-training: self-supervised generation of text (i.e., next-word predictions) on massive web scrape data sets

lightweight PyTorch re-implementation of GPT: [minGPT](#)

more powerful: [nanoGPT](#), [LitGPT](#)



[source](#)

GPT Evolution

GPT: discriminative fine-tuning on specific tasks (e.g., summarization, translation, question-answering) with much smaller data sets

GPT-2, GPT-3 (175 billion parameters): also zero- or few-shot learning

GPT-4: extend to multimodal model (image and text inputs, text outputs)
capabilities

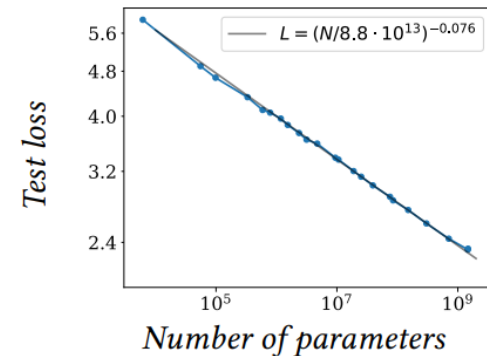
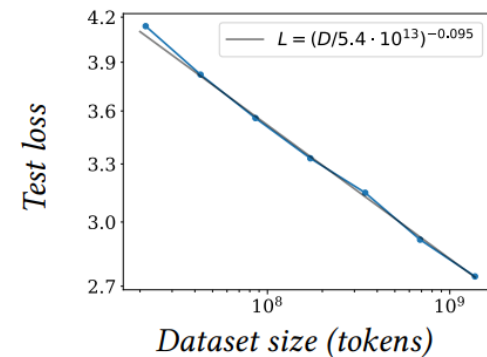
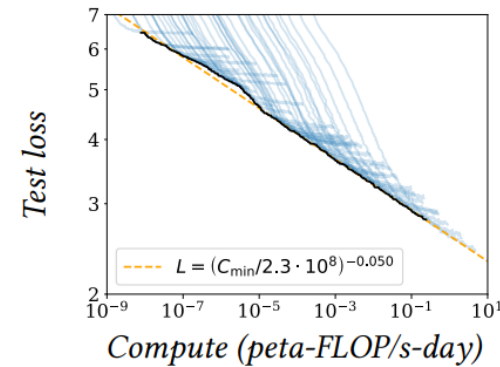
Size Matters: **LARGE** Language Models

scaling laws, Chinchilla: coupled performance power laws with model size, amount of training data, and compute

→ era of large-scale models

emergent abilities of LLMs:

- multi-task learning: perform new tasks at test time without task-specific training (via prompting)
- reasoning capabilities



Some LLM Numbers

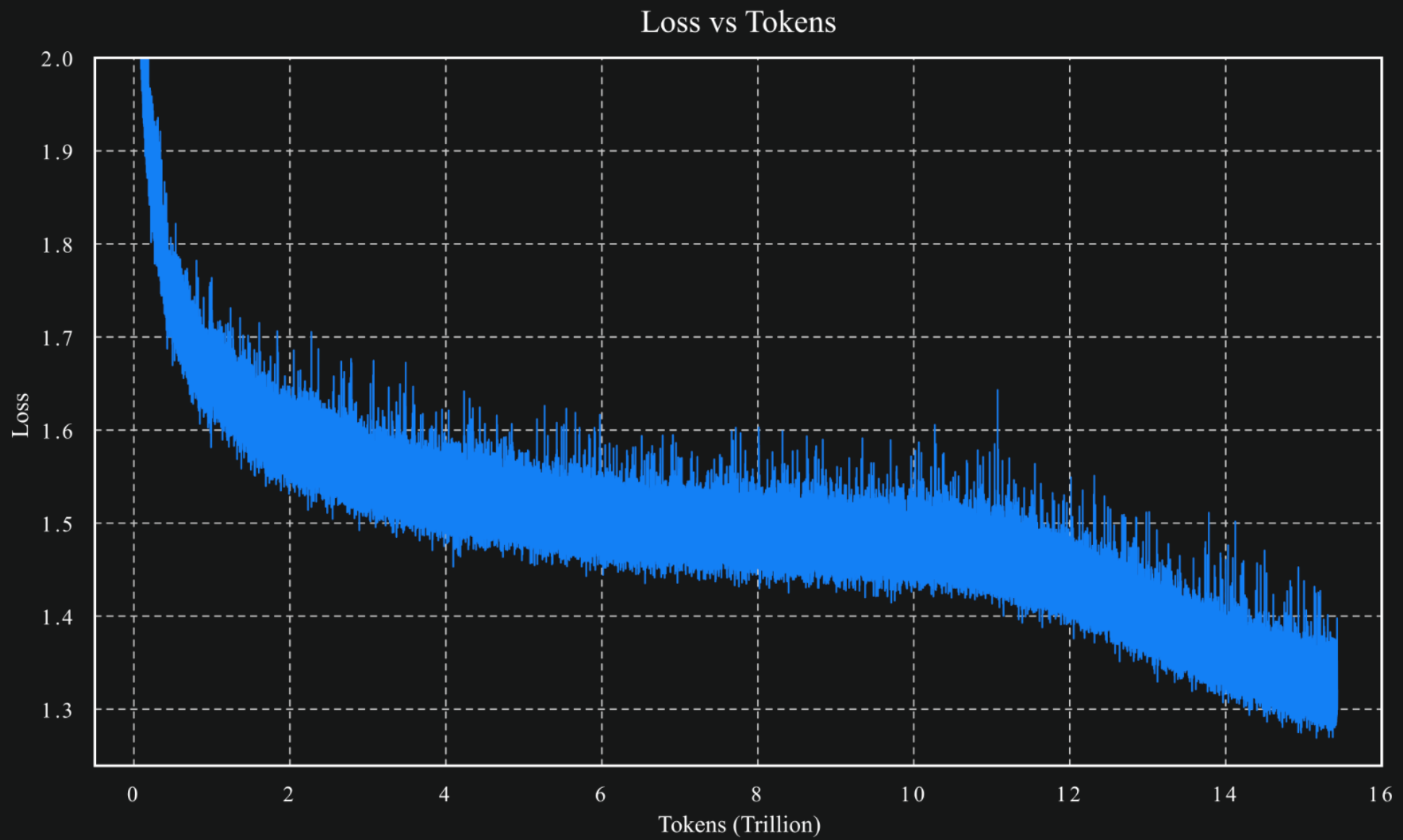
example [Kimi K2](#) (Mixture-of-Experts model):

- vocabulary size (tokens): 160K
- embedding dimension: 7,168
- context length (tokens): 128K
- total parameters: 1T
- training tokens: 15.5T
- number of layers: 61
- number of attention heads: 64
- number of experts: 384

as usual, comes in base and instruct variants

← → a lot of memorizing

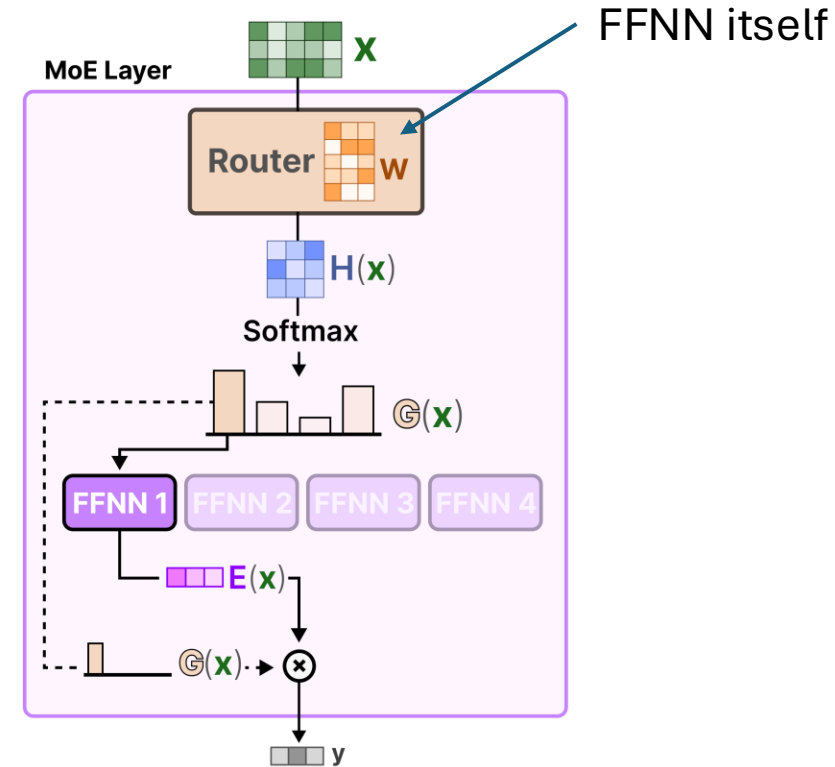
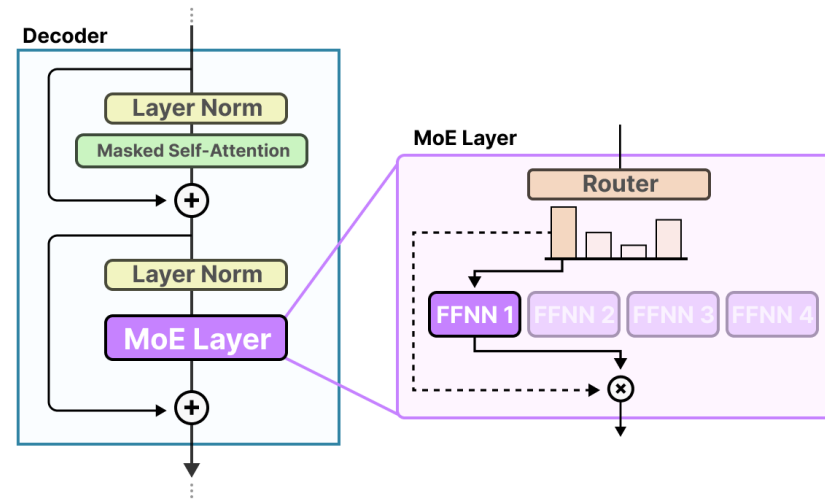
compression of the internet



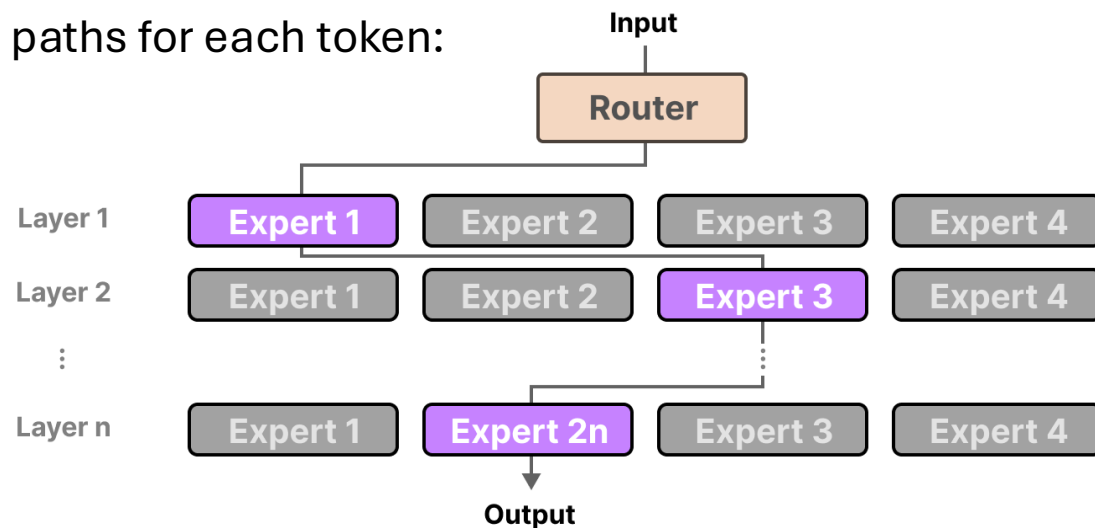
[source](#)

Mixture-of-Experts (MoE)

idea: replace big FFNN
with several smaller ones
and activate only few
→ less active parameters



different paths for each token:



example (actually more abstract):



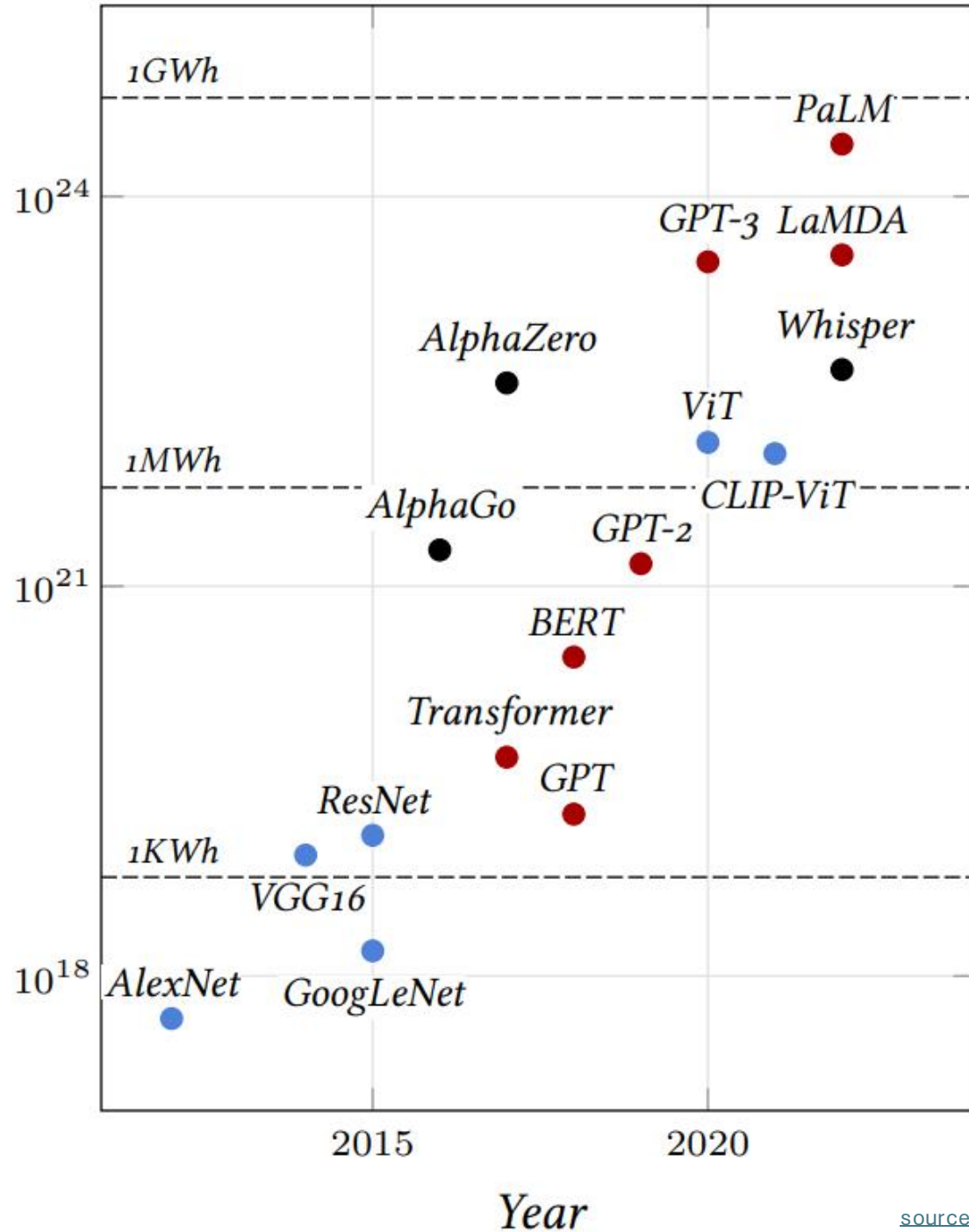


 Claude

 Gemini

 LLaMA

Training cost (FLOP)



[source](#)

Llama 3

Closed-source vs. open-weight models

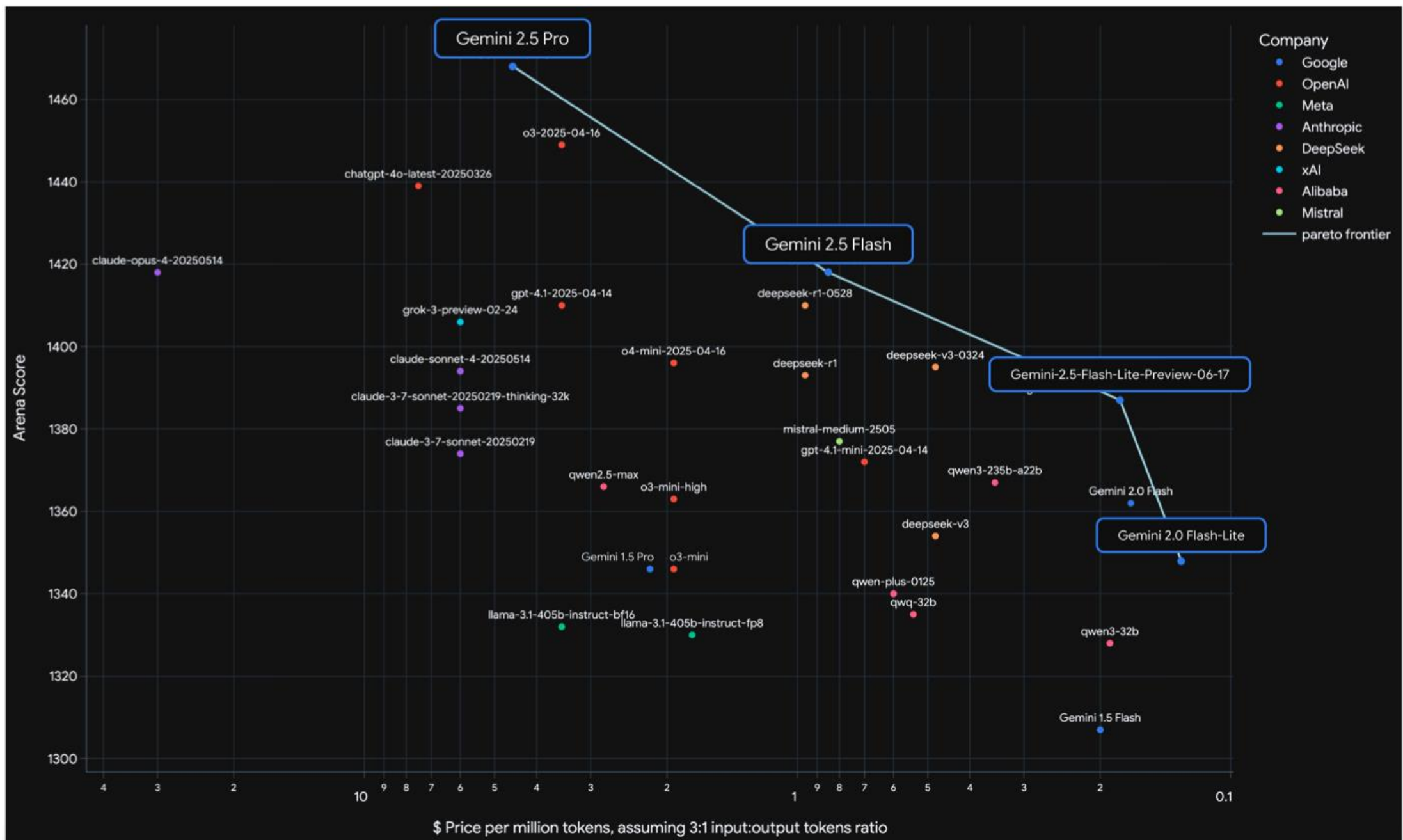
Llama 3.1 405B closes the gap with closed-source models for the first time in history.

@maximelabonne



LMarena

Rank (UB) ↑	Model ↓	Score ↓	95% CI (±) ↓	Votes ↓	Organization ↓	License ↓
1	 gemini-2.5-pro	1455	±5	41.731	Google	Proprietary
1	 claude-opus-4-1-20250805-thinking-16k	1451	±6	11.750	Anthropic	Proprietary
2	 o3-2025-04-16	1444	±4	43.898	OpenAI	Proprietary
2	 gpt-5-high	1442	±6	15.076	OpenAI	Proprietary
2	 chatgpt-4o-latest-20250326	1441	±4	36.426	OpenAI	Proprietary
3	 gpt-4.5-preview-2025-02-27	1439	±6	15.271	OpenAI	Proprietary
3	 claude-opus-4-1-20250805	1438	±6	18.341	Anthropic	Proprietary
5	 gpt-5-chat	1430	±6	11.808	OpenAI	Proprietary
6	 qwen3-max-preview	1428	±7	8.781	Alibaba	Proprietary
8	 grok-4-0709	1422	±5	21.446	xAI	Proprietary
8	 claude-opus-4-20250514-thinking-16k	1420	±5	27.351	Anthropic	Proprietary
8	 kimi-k2-0711-preview	1420	±5	23.527	Moonshot	Modified MIT
8	 qwen3-235b-a22b-instruct-2507	1420	±6	17.947	Alibaba	Apache 2.0
8	 deepseek-v3.1-thinking	1419	±8	8.226	DeepSeek	MIT
8	 deepseek-v3.1	1418	±7	9.822	DeepSeek	MIT
8	 deepseek-r1-0528	1418	±6	20.863	DeepSeek	MIT
8	 kimi-k2-0905-preview	1418	±11	2.918	Moonshot	Modified MIT
10	 glm-4.5	1411	±6	14.105	Z.ai	MIT



[source](#)

Another Trend: Small Language Models (SLM)

most powerful LLMs keep getting bigger (> 1 trillion parameters)

[GPT-4o](#), [Claude 3.5](#), [Gemini 1.5](#), ...

but slimmed-down (yet still strong) versions get smaller

[Gemma](#) (2B), [Phi-3](#) (3.8B), [Mistral 7B](#), [Llama3](#) (8B), ...

can run (inference) on laptops, smartphones, etc.

easier fine-tuning → specific tasks

Modern Language Models in a Nutshell

tokenization (split text into chunks)

pretraining of transformer (next- or masked-token prediction, self-supervised)

- vector embeddings from tokens (semantic meaning)
- add positional encoding to embeddings (use order of sequence, as attention is permutation-invariant)
- attention mechanism → contextual embeddings

finetuning (specialization)

- for specific task (encoder LLM)
- instruction tuning (decoder LLM), potentially followed by alignment

discussed so far